

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 06 | Recursion, Backtracking and Divide & Conquer

Week 6 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 06 at a glance

Item	Details
Lab title	Recursion, Backtracking and Divide & Conquer
Week	6 of 16
Prerequisites	Lab 02 (Big-O), Lab 04 (Binary Search)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab06_Instructor.ipynb , Lab06_Student.ipynb
Next lab	Lab 07: Efficient Sorting (Merge Sort, Quick Sort, Counting Sort)

1. Learning Objectives

By the end of this lab, you will be able to:

1. Define recursion and identify the **base case** and the **recursive case** of a problem.
2. Trace recursive calls using the **call stack** and a **recursion tree**.
3. Write recursive solutions for classic problems: factorial, power, sum of digits, binary search, Tower of Hanoi.
4. Explain why naive recursive Fibonacci is $O(2^n)$ and fix it with **memoization**.
5. Convert between recursion and iteration, and know Python's recursion limit.
6. Use **backtracking** to generate subsets and permutations.
7. Explain the **divide & conquer** strategy that powers merge sort and quick sort.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-2	Implement recursive and divide-and-conquer algorithms	Apply (C3)
CLO-3	Analyse recursive algorithms using recursion trees	Analyse (C4)

2. Motivation: Solving Big Problems with Smaller Copies

Some problems are naturally made of **smaller versions of themselves**. A folder contains files and other folders, which contain files and other folders. A family tree has parents, who have parents. **Recursion** is a way to solve such problems by letting a function **call itself** on a smaller input.

Analogy: Russian dolls (Matryoshka)

To find the smallest doll, you open the big doll and repeat the **same action** on the doll inside. You stop when a doll cannot be opened (the **base case**). That is recursion.

2.1 Recursion in Daily Life and Computing

Example	Recursive structure
Folders on your computer	A folder contains folders → search all files recursively
Family tree / organisation chart	Each person has children who have children

Example	Recursive structure
Two mirrors facing each other	An image inside an image inside an image...
Fractals (ferns, snowflakes, coastlines)	Each part looks like the whole
HTML pages	Tags inside tags (a <code>div</code> inside a <code>div</code>)
Mathematics: $n! = n \times (n-1)!$	Defined using a smaller version of itself

2.2 Recursion in Industry

- **File systems and compilers** walk tree structures recursively (e.g., `os.walk`, parsing code into syntax trees).
- **Fast algorithms** such as Merge Sort, Quick Sort, Binary Search and the Fast Fourier Transform are built on divide & conquer.
- **Games and puzzles** (chess engines, Sudoku solvers) use recursive search with backtracking.
- **Web crawlers** visit a page, then recursively visit the pages it links to.

2.3 Recursion in Research

- **AI planning and game playing:** Minimax and Monte Carlo Tree Search (used in AlphaGo) explore game trees recursively.
- **Dynamic programming**, central to bioinformatics (DNA alignment) and reinforcement learning, starts from recursive formulas plus memoization.
- **Recursive neural networks** process sentences as trees in NLP.

2.4 Why It Matters in 2026–2030

Thinking recursively is a superpower

Trees, graphs, parsing and search, the backbone of AI systems, databases and compilers, are recursive by nature. Once you learn to **trust the recursion** (assume the smaller call works and focus on one step), complex problems become short, elegant programs. The rest of this course (trees, BST, heaps, graphs) depends heavily on recursion.

3. Part A: Anatomy of a Recursive Function

Every correct recursive function has **two parts**:

Part	Role	Factorial example
Base case	The smallest input, answered directly; it stops the recursion	$0! = 1$
Recursive case	Reduces the problem and calls itself on a smaller input	$n! = n \times (n - 1)!$

```
def factorial(n):
    if n == 0:                # base case
        return 1
    return n * factorial(n - 1) # recursive case (smaller input)

print(factorial(5))
```

⚠ No base case = infinite recursion

If the base case is missing or never reached, the function calls itself forever until Python stops it with `RecursionError: maximum recursion depth exceeded`.

3.1 The Call Stack

Each function call gets a **stack frame** that stores its own local variables. Calls are **pushed** onto the call stack and **popped** when they return (Last In, First Out, like the Stack you will build in Lab 10).

Winding (going down) and unwinding (coming back)

```
factorial(4)
-> 4 * factorial(3)
    -> 3 * factorial(2)
        -> 2 * factorial(1)
            -> 1 * factorial(0)
                -> 1                (base case)
            <- 1 * 1 = 1
        <- 2 * 1 = 2
    <- 3 * 2 = 6
<- 4 * 6 = 24
```

See the call stack in action

```
def factorial_trace(n, depth=0):
    indent = "    " * depth
    print(f"{indent}factorial({n}) called")
    if n == 0:
        print(f"{indent}base case -> 1")
        return 1
    result = n * factorial_trace(n - 1, depth + 1)
    print(f"{indent}factorial({n}) returns {result}")
    return result

factorial_trace(4)
```

Space complexity of recursion includes the call stack: `factorial(n)` keeps n frames at the deepest point, so it uses $O(n)$ space.

3.2 How to Think Recursively (The Leap of Faith)

1. **Define** exactly what the function returns, e.g., “`sum_list(a)` returns the sum of list `a`”.
2. **Base case:** what is the answer for the smallest input (empty list $\rightarrow 0$)?
3. **Trust** that the function already works for a smaller input (`sum_list(a[1:])`).
4. **Combine:** use that smaller answer to build the full answer (`a[0] + sum_list(a[1:])`).

```
def sum_list(a):
    if not a:
        return 0
    return a[0] + sum_list(a[1:]) # first element + sum of the rest

def count_down(n):
    if n == 0:
        print("Go! 🚀")
        return
    print(n)
    count_down(n - 1)

print(sum_list([4, 8, 15, 16, 23, 42]))
count_down(3)
```



4. Part B: Classic Recursive Problems

4.1 Power: x^n

```
def power(x, n):                                # O(n)
    if n == 0:
        return 1
    return x * power(x, n - 1)

def fast_power(x, n):                            # O(log n): halve the exponent
    if n == 0:
        return 1
    half = fast_power(x, n // 2)
    return half * half if n % 2 == 0 else half * half * x

print(power(2, 10), fast_power(2, 10), fast_power(3, 13) == 3 ** 13)
```

4.2 Recursive Binary Search

```
def binary_search_rec(arr, target, low, high):
    if low > high:                                # base case: empty range
        return -1
    mid = (low + high) // 2
    if arr[mid] == target:
        return mid
    if arr[mid] < target:
        return binary_search_rec(arr, target, mid + 1, high)
    return binary_search_rec(arr, target, low, mid - 1)

data = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
print(binary_search_rec(data, 72, 0, len(data) - 1))
```

4.3 Tower of Hanoi

Move n discs from peg A to peg C using peg B, moving one disc at a time and never placing a bigger disc on a smaller one. The recursive insight: **move $n-1$ discs out of the way, move the biggest disc, then move the $n-1$ discs on top of it.**

```
def hanoi(n, source, helper, target):
    if n == 0:
        return 0
    moves = hanoi(n - 1, source, target, helper) # move n-1 to helper
    print(f"Move disc {n} from {source} to {target}")
    moves += 1
    moves += hanoi(n - 1, helper, source, target) # move n-1 onto target
    return moves

print("Total moves:", hanoi(3, "A", "B", "C"))
```

The number of moves is $2^n - 1$. For 64 discs (the legend), that is about 1.8×10^{19} moves: at one move per second it would take around 585 billion years. A perfect example of **exponential** growth.



5. Part C: Recursion Trees and Memoization

5.1 Fibonacci: The Danger of Repeated Work

Fibonacci numbers: $F(0) = 0$, $F(1) = 1$, $F(n) = F(n-1) + F(n-2)$. The direct recursive code is beautiful but **extremely slow**.

Recursion tree for fib(5)

```

fib(5)
├── fib(4)
│   ├── fib(3)
│   │   ├── fib(2) — fib(1), fib(0)
│   │   └── fib(1)
│   └── fib(2) — fib(1), fib(0)
└── fib(3)
    ├── fib(2) — fib(1), fib(0)
    └── fib(1)

```

<- fib(2) computed again
 <- whole fib(3) computed again!

```

calls = 0
def fib_naive(n):
    global calls
    calls += 1
    if n < 2:
        return n
    return fib_naive(n - 1) + fib_naive(n - 2)

for n in [10, 20, 25]:
    calls = 0
    value = fib_naive(n)
    print(f"fib({n}) = {value:>6}  ->  {calls:>7,} function calls")

```

The number of calls roughly **doubles** with each increase of n : the complexity is $O(2^n)$. `fib_naive(50)` would need about 40 billion calls.

5.2 Memoization: Remember What You Already Computed

Memoization stores the result of each call in a dictionary. Before computing, check whether the answer is already known. Each $F(k)$ is computed **once**, so the complexity drops to $O(n)$.

```
def fib_memo(n, memo=None):
    if memo is None:
        memo = {}
    if n in memo:
        return memo[n]          # already computed: O(1)
    if n < 2:
        return n
    memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]

from functools import lru_cache

@lru_cache(maxsize=None)        # Python's built-in memoization
def fib_cached(n):
    return n if n < 2 else fib_cached(n - 1) + fib_cached(n - 2)

print(fib_memo(80), fib_cached(80))
```

5.3 Recursion vs Iteration

```
def fib_iter(n):                # O(n) time, O(1) space, no recursion
    limit
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a

import sys
print(fib_iter(80), "| Python recursion limit:", sys.getrecursionlimit())
```

	Recursion	Iteration (loops)
Code	Short, close to the mathematical definition	Sometimes longer
Memory	O(depth) for the call stack	Usually O(1)
Risk	Stack overflow / RecursionError (~1000 deep in Python)	Infinite loop
Best for	Trees, graphs, divide & conquer, backtracking	Simple repetition, very deep processes



6. Part D: Backtracking

Backtracking builds a solution step by step, explores one choice, and **undoes** (backtracks) that choice to try the next one. It systematically explores all possibilities.

Generate all subsets (e.g., all possible combinations of toppings)

```
def subsets(items):
    result = []
    current = []

    def explore(i):
        if i == len(items):                # decided for every item
            result.append(current[:])
            return
        current.append(items[i])           # choice 1: include items[i]
        explore(i + 1)
        current.pop()                      # undo (backtrack)
        explore(i + 1)                     # choice 2: exclude items[i]

    explore(0)
    return result

print(subsets(["cheese", "olives", "chicken"]))
```

Each item has 2 choices, so n items give 2^n subsets. Backtracking is exponential, but it is often the only exact method, and it can be sped up by **pruning** branches that cannot lead to a solution (e.g., in Sudoku).



7. Part E: Divide & Conquer

Divide & conquer solves a problem in three steps:

1. **Divide** the problem into smaller sub-problems (usually halves).
2. **Conquer** each sub-problem recursively.
3. **Combine** the sub-answers into the final answer.

```
def max_dc(a, low, high):
    if low == high:                        # one element
        return a[low]
    mid = (low + high) // 2
    left_max = max_dc(a, low, mid)         # conquer left half
    right_max = max_dc(a, mid + 1, high)   # conquer right half
    return left_max if left_max > right_max else right_max  # combine

marks = [67, 89, 45, 92, 78, 55, 81]
print(max_dc(marks, 0, len(marks) - 1))
```

Algorithm	Divide	Combine	Complexity
Binary search	Pick one half	Nothing	$O(\log n)$
Max by D&C	Two halves	Compare two values	$O(n)$
Merge sort (Lab 07)	Two halves	Merge two sorted halves: $O(n)$	$O(n \log n)$
Quick sort (Lab 07)	Partition around a pivot	Nothing	$O(n \log n)$ average

8. C++ Corner

C++

```
#include <iostream>
#include <unordered_map>
using namespace std;

long long factorial(int n) {
    if (n == 0) return 1;           // base case
    return n * factorial(n - 1);    // recursive case
}

unordered_map<int, long long> memo;
long long fib(int n) {
    if (n < 2) return n;
    if (memo.count(n)) return memo[n];
    return memo[n] = fib(n - 1) + fib(n - 2);
}

void hanoi(int n, char from, char via, char to) {
    if (n == 0) return;
    hanoi(n - 1, from, to, via);
    cout << "Move disc " << n << " from " << from << " to " << to << endl;
    hanoi(n - 1, via, from, to);
}

int main() {
    cout << factorial(10) << " " << fib(80) << endl;
    hanoi(3, 'A', 'B', 'C');
}
```

9. Common Mistakes

Mistake	Effect	Fix
Missing base case	RecursionError	Always write the base case first
Input does not get smaller	Infinite recursion	Make sure each call moves towards the base case
Forgetting <code>return</code> before the recursive call	Function returns <code>None</code>	<code>return f(n - 1)</code>
Repeated sub-problems without memo	Exponential time	Use a dictionary or <code>@lru_cache</code>
Using a mutable default <code>memo={}</code>	Memo shared between unrelated calls	Use <code>memo=None</code> then create it
Very deep recursion in Python	Crashes around depth 1000	Use iteration for long linear processes

10. Lab Tasks

Solve all tasks in `Lab06_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Sum of Digits

Write a **recursive** function `digit_sum(n)` that returns the sum of the digits of a non-negative integer. No loops, no `str()`.

```
digit_sum(2026) → 10
```

Task 2 Reverse a String Recursively

Write `reverse(s)` recursively, without slicing tricks like `[::-1]` and without loops. Hint: reverse of `s` = reverse of `s[1:]` + `s[0]`.

```
reverse("0kara") → "arak0"
```

Task 3 Count Files in a Folder Tree

A folder is represented as a dictionary: keys are names; a value of `None` means a file, and a dictionary means a sub-folder. Write `count_files(folder)` and `total_depth(folder)` (the maximum nesting depth; a folder with only files has depth 1).

This is exactly how your operating system walks directories.

```
{"notes.txt": None, "labs": {"lab1.py": None, "data": {"a.csv": None}}}
count_files → 3, total_depth → 3
```

Task 4 Grid Paths with Memoization

A delivery rider starts at the top-left of an $r \times c$ grid of streets and can move only **right** or **down**. Some cells are blocked (`1`). Count the number of paths to the bottom-right.

Write the recursive solution `paths(grid)`, then add memoization so that a 15×15 grid finishes instantly.

```
grid = [[0,0,0],
        [0,1,0],
        [0,0,0]] → 2
```

Task 5 Tower of Hanoi Move List

Write `hanoi_moves(n, a, b, c)` that **returns** the list of moves as tuples (`disc`, `from`, `to`) instead of printing. Verify that the length is $2^n - 1$ and that no move ever places a bigger disc on a smaller one (write a simulator).

```
hanoi_moves(2, "A", "B", "C") → [(1, "A", "B"), (2, "A", "C"), (1, "B", "C")]
```

Task 6 Permutations (Backtracking)

Write `permutations(items)` that returns all orderings of a list using backtracking with a `used` list. Do not use `itertools`. The result order should be lexicographic if the input is sorted.

```
permutations([1, 2, 3]) → [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]
```

Task 7 Flood Fill: Count Lakes on a Map

A satellite map is a grid where `1` is water and `0` is land. Water cells connected **up, down, left or right** form one lake. Write `count_lakes(grid)` using a recursive `fill(i, j)` that marks a whole lake as visited.

This is the same algorithm used by the paint-bucket tool in MS Paint and by region detection in remote-sensing images.

```
[[1,1,0,0],
 [1,0,0,1],
 [0,0,1,1],
 [0,0,0,0]] → 2
```

11. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub `DSA-Labs/Lab06` folder.

#	Problem	Level	Concept practised
509	Fibonacci Number	Easy	Recursion + memoization
344	Reverse String	Easy	Recursion with two pointers
231	Power of Two	Easy	Recursive halving
70	Climbing Stairs	Easy	Recursion → memoization

#	Problem	Level	Concept practised
50	Pow(x, n)	Medium	Fast power, divide & conquer
78	Subsets	Medium	Backtracking
46	Permutations	Medium	Backtracking (Task 6)
200	Number of Islands	Medium	Flood fill (Task 7)



12. Home Task

1. Finish all tasks and push `Lab06_Student.ipynb` to GitHub.
2. Write recursive functions for: palindrome check, GCD (Euclid's algorithm), and printing all binary strings of length n.
3. Draw the recursion tree of `fib_naive(6)` on paper and count the calls. Then explain how memoization changes the tree.
4. **Mini-project: Sudoku Solver.** Write a backtracking solver for a 9×9 Sudoku. Count how many recursive calls it makes. Try to reduce them with pruning (check row, column and box before placing a digit).
5. Solve at least 3 LeetCode problems from the list.

? 13. Viva Questions

1. What is recursion? What are the base case and recursive case?
2. What happens if a recursive function has no base case?
3. Explain the call stack using `factorial(3)`.
4. What is the space complexity of a recursive function with depth n?
5. Why is naive recursive Fibonacci $O(2^n)$?
6. What is memoization? How does it change Fibonacci's complexity?
7. When would you choose iteration over recursion?
8. How many moves does Tower of Hanoi need for n discs?
9. What is backtracking? How many subsets does a set of n items have?
10. Explain the three steps of divide & conquer with an example.



14. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain

Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.



15. References and Further Learning

- `functools.lru_cache`: docs.python.org/3/library/functools.html
- Step through recursive calls visually at pythontutor.com
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).
- Visualise code step by step: pythontutor.com; animated data structures: visualgo.net.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.